# HippoMem: A Practical Memory Architecture for AI Applications

**Author:** Ashish Mathew **Version:** 0.8 **Date:** March 2026

## Disclaimer

hippomem is an actively evolving open-source project. This document reflects the architecture and design intent of the current version, not a final or frozen specification. The system is functional and being actively used, but you should expect rough edges — in the implementation, in the documentation, and in this paper itself. That is expected and welcomed at this stage.

If you find gaps, inaccuracies, or improvements, please raise a GitHub issue or open a pull request. Contributions from developers, researchers, and end users will directly shape what gets built next. This document will be updated as the system evolves and new capabilities are added. Readers will be notified of meaningful changes through the project's release notes and changelog.

hippomem is being built openly. The author believes that persistent, evolving memory is a foundational capability for AI systems, and that it should be built in public — with the community, not behind closed doors.

## Abstract

hippomem is a standalone memory layer for LLM applications that treats memory as a lifecycle rather than a lookup. It sits alongside any AI application — without replacing the model or the infrastructure around it — and gives it persistent, structured, evolving memory. Every interaction passes through three phases: memories are retrieved before the LLM call, updated after it, and maintained over time through a consolidation process. The system builds memory across three dimensions — episodic events (what was discussed), entity profiles (what is known about people, projects, and organizations), and user identity (what is known about the user themselves) — and connects them through a typed graph that makes memory not just storable but navigable.

Integration requires four operations: `decode()` before the LLM call to retrieve and synthesize relevant context, `encode()` after it to decide how the completed turn should change stored memory, `consolidate()` for periodic maintenance — compressing episode update histories, enriching entity profiles, and synthesizing a user persona — and `retrieve()` for direct programmatic search that returns raw structured results independent of the LLM injection path. Relevance decay and working memory demotion are continuous processes that run during encoding, not periodic steps.

This paper describes the architecture in full: the retrieval cascade that selects relevant context efficiently across three escalating stages, the encoding logic that decides how each interaction should affect stored memory, the consolidation process that keeps memory coherent over time, the memory graph and its role in surfacing connected context, and the operational layer that makes all of it inspectable and controllable in real deployments. Unlike a fact store, hippomem does not attempt to record everything — selective forgetting is a first-class design property, not a limitation.

## 1. What hippomem Proposes

hippomem is a standalone memory layer for AI applications. It is not a model, not a new inference architecture. It is infrastructure — a subsystem that handles memory so your application does not have to.

Integration is four function calls:

```
# Before calling your LLM:
result = await memory.decode(user_id, user_message, conversation_history=history)
# result.context → inject into your LLM system prompt

# After your LLM responds:
await memory.encode(user_id, user_message, assistant_response, decode_result=resu
        lt)

# Periodic maintenance (episode compression + entity enrichment + persona):
await memory.consolidate(user_id)

# Direct structured search, independent of the decode/encode lifecycle:
result = await memory.retrieve(user_id, query, mode="hybrid", top_k=5)
for episode in result.episodes:
    print(episode.core_intent, episode.entities, episode.related_episodes)
```

The central design idea is that memory is a lifecycle, not a lookup. Every interaction passes through retrieval, update, and eventual consolidation. Memories are reinforced when used, branched when topics diverge, created when something new and significant is said, and gradually deprioritized when they become stale. At the same time, the system builds up structured knowledge about the entities a user talks about — people, projects, organizations — and about the user themselves. These representations enrich future retrieval in ways that pure episodic search cannot.

The rest of this paper describes how each part of this works.

---

## 2. Memory Representations

### 2.1 Engrams: The Unit of Memory

The fundamental unit of stored memory in hippomem is the **Engram** — a term borrowed from neuroscience, where it refers to the physical substrate of a memory trace in the brain. Every piece of information hippomem stores is an engram of one of four kinds:

- **Episode** — a conversational memory event. It has a topic (`core_intent`) and an evolving list of facts or updates about that topic. Episodes are the primary memory type and are what the retrieval cascade surfaces.
- **Summary** — a condensed representation of multiple episodes. Planned for a future version; the data model supports it today.
- **Entity** — a named-entity profile. A person, organization, project, place, or pet that has been mentioned in conversation. Entities accumulate structured facts across interactions.
- **Persona** — a synthesized narrative of the user's stable identity. Generated by consolidation from accumulated self-traits. Used to make retrieval context identity-aware.

Each engram carries a `core_intent` (the primary text content — topic sentence for episodes, canonical name for entities), two fact-holding fields, a `relevance_score` (starts at 1.0, decays with time), and a `reinforcement_count` (how many times it has been actively used in synthesis).

The two fact-holding fields separate what has been consolidated (`updates`) from what has arrived since (`pending_facts`). How and when these are merged is covered in Section 6.

> ***In plain terms:*** *An engram is hippomem's word for a memory. Every memory has a topic, a set of facts, and a score that tells the system how relevant it currently is. Different kinds of memories — conversational events, people, projects, and your own identity — are stored in the same structure but serve different roles.*

## 2.2 Working Memory: Active and Dormant Tiers

Not all stored memories are equally accessible. hippomem maintains a **working memory** — a tiered structure that tracks which engrams are currently in focus.

**Active engrams** (up to 5 by default) are the memories the system is currently paying attention to. They are ordered by recency of use: the most recently engaged memory sits at position 0. When the retrieval cascade runs, it checks active engrams first, before doing any broader search.

**Dormant engrams** (up to 5 by default) are memories that were recently active but have since been displaced. They remain quickly accessible — included in the local scan — without occupying the primary focus.

Together, active and dormant form the "local neighborhood" that the retrieval cascade checks before escalating to any broader search. This is the critical optimization for interactive systems: most turns can be resolved using memories that are already close to hand.

When active capacity is exceeded, the oldest active memory is pushed to dormant (FIFO). When dormant capacity is exceeded, the oldest dormant memory is evicted from fast access entirely — it remains in long-term storage and is retrievable through the full search path, but it no longer receives preferential attention.

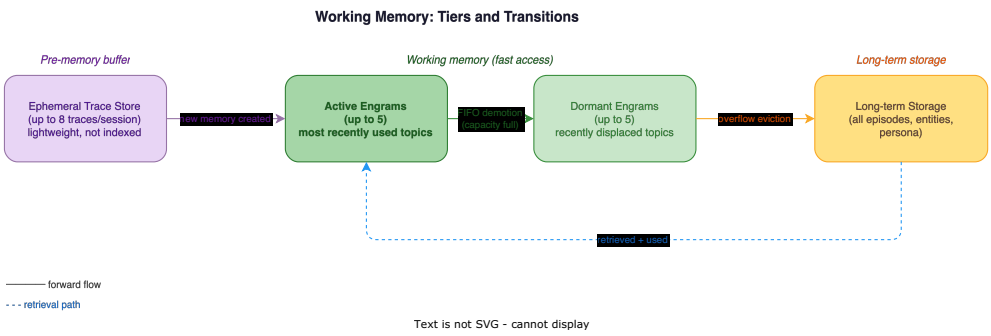



Figure 1 — Working memory tiers and transitions

*Figure 1 — Working memory tiers and transitions (simplified)*

> ***In plain terms:*** *Think of active memories as what is currently on your desk, and dormant memories as what is still within arm's reach on the shelf behind you. Everything else is in the filing cabinet. When you need something, you check the desk first, then the shelf, and only pull out the filing cabinet if neither has what you need.*

## 2.3 The Ephemeral Trace Store

Not every conversational turn deserves to become a permanent memory. A casual greeting, a short clarifying question, a transactional exchange — these may not warrant the cost of creating and maintaining a full engram.

The **Ephemeral Trace Store (ETS)** is a lightweight pre-memory buffer. When the system decides that a turn is not significant enough to create a full memory, but is also not so trivial that it should be ignored entirely, it stores a short text summary as a *trace* instead. Traces are kept in a fixed-capacity FIFO buffer (8 per session by default). They are not embedded, not indexed, and not part of the scored retrieval path.

Traces serve one purpose: they provide context to future memory-creation decisions. When the system later encounters a turn that does merit a full memory, the existing traces from that session are included in the creation prompt — giving the LLM a richer picture of what has been discussed, even though those prior turns did not individually cross the memory threshold.

> ***In plain terms:*** *Not everything you say in a conversation needs to be remembered forever. The trace store is hippomem's notepad — a temporary holding area for things that might matter later. If a subsequent turn turns out to be significant, the traces inform it. If nothing significant follows, the traces quietly cycle out.*

### 2.4 Entity Memory

When people, projects, organizations, places, pets, tools, or other named entities are mentioned in conversation, hippomem creates and maintains structured profiles for them as Entity engrams. This is meaningfully different from episodic memory: where an episode captures *what happened* in a conversation, an entity captures *what is known* about a persistent named object in the user's world.

Entity profiles accumulate facts across interactions. The first time a user mentions their colleague Sarah, a profile is created with whatever facts were extractable from that turn. The next time Sarah comes up, new facts are appended to her profile and her `reinforcement_count` is incremented. Over time, the profile becomes richer.

hippomem recognizes six entity types: person, organization, place, project, pet, tool, and other.

New facts accumulate in the entity's pending buffer during encoding and are periodically merged into a clean consolidated profile during consolidation. Between merges, all read paths combine both — so the full picture is always visible. How this merge works is covered in Section 6.

Entity identity is maintained carefully. When the same entity is mentioned under a different name or in a different phrasing, hippomem applies a lookup strategy to determine whether to update an existing entity or create a new one (described in detail in Section 5.4).

> ***In plain terms:*** *If you keep mentioning your colleague Sarah, hippomem builds and refines a profile of Sarah over time — her role, her project, her relationship to your work. Future conversations that involve Sarah can draw on everything already known about her, without you having to re-explain who she is every time.*

### 2.5 Self Memory and Persona

In addition to what users talk *about*, hippomem tracks things that users implicitly reveal *about themselves*. Preferences, work habits, communication style, goals, constraints, ongoing projects — these stable signals are captured as **self-traits** and stored independently of any specific episodic memory.

Self-traits are extracted quietly, turn by turn. The LLM is shown the user's current active trait profile and asked to return only signals that are genuinely new or meaningfully changed — not things already captured. High-confidence signals (≥0.8) activate immediately on first observation. Lower-confidence signals are stored but remain inactive until a second independent observation confirms them. Confidence and evidence count accumulate over repeated observations.

Deactivation is the exclusive responsibility of the consolidation layer: traits that have gone unobserved for more than 30 days, with low evidence and low confidence, are conservatively deactivated. A high-

confidence or frequently-reinforced trait is never pruned. A deactivated trait is immediately reactivated the next time it is observed.

During consolidation, active high-confidence self-traits are synthesized into a single **Persona engram** — a short narrative that summarizes who the user is. This persona is injected into the synthesis prompt during `decode()`, enabling identity-aware context generation. The LLM synthesizing memory context is not working from a blank slate; it knows something about the person it is producing context for.

> ***In plain terms:*** *Over time, hippomem builds a picture of who you are — your preferences, your work context, your communication style — from things you say naturally, without requiring you to explicitly "tell" the system anything. This becomes part of how it interprets and presents memory, making responses progressively more personalized.*

---

## 3. decode(): Memory Recall and Context Synthesis

Before the application calls its LLM, it calls `decode()`. This is the retrieval phase: hippomem selects the memories most relevant to the current moment and synthesizes them into a compact memory context for injection into the system prompt.

The central design principle of the `decode()` path is **selective escalation**: start with the cheapest possible retrieval path, and only escalate to more expensive paths when cheaper ones are insufficient. This reflects the reality of conversational systems — the majority of turns continue an ongoing thread, and a system that performs full retrieval on every turn pays a cost that most turns do not require.

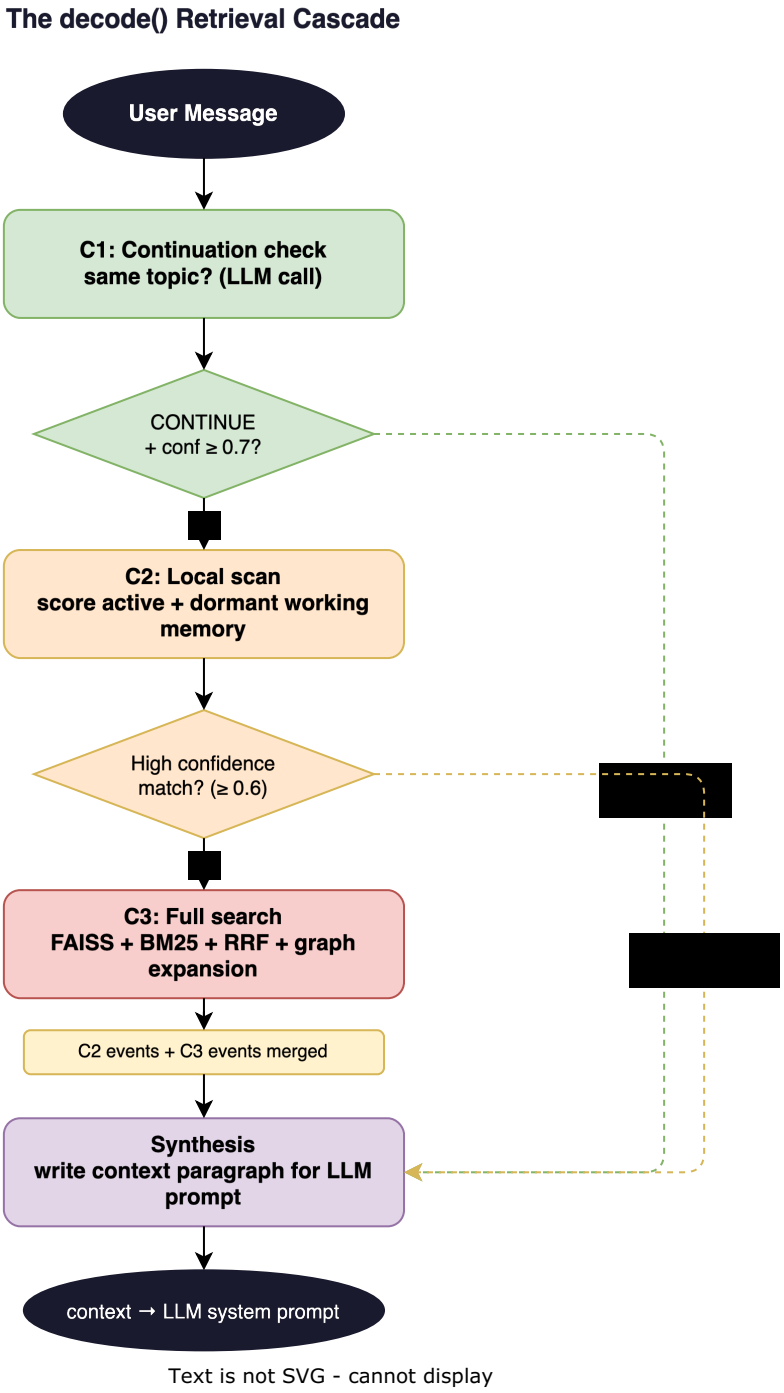



Figure 3 — The decode() retrieval cascade

*Figure 3 — The decode() retrieval cascade (simplified)*

### 3.1 The Cascade Architecture

The retrieval cascade has three stages, each representing a different scope and cost of search:

- **C1** checks whether the current message continues the active topic, using a single fast LLM call. If the answer is a confident yes, retrieval ends here.
- **C2** scores the active and dormant working memories against the current query using a composite scoring formula. If a confident match exists within working memory, retrieval ends here.

- **C3** performs full long-term search: dense vector retrieval (FAISS), keyword retrieval (BM25), rank fusion, and graph expansion over connected memories. This is triggered only when C1 and C2 together do not yield sufficient confidence.

The cascade exits as early as possible. In practice, a well-functioning system will resolve the majority of turns at C1 or C2 — the expensive C3 path is reserved for topic shifts, cold starts, and queries that require context the current working memory does not hold.

### 3.2 C1: The Continuation Check

The first stage is a single LLM call that asks one question: does this message continue the currently active topic, shift to something new, or is the situation uncertain?

The LLM is given the current message, the last two turns of conversation history, and the `core_intent` of the currently active memory. It returns a `decision` — one of `CONTINUE`, `SHIFT`, or `UNCERTAIN` — and a `confidence` score between 0 and 1.

If the decision is `CONTINUE` and confidence meets or exceeds the threshold (default 0.7), the cascade exits immediately. The active memory is used as-is, with no further retrieval. Both the C2 embedding step and the full C3 search are skipped entirely.

If the decision is anything else — `SHIFT`, `UNCERTAIN`, or a low-confidence `CONTINUE` — the cascade proceeds to C2.

The C1 check is deliberately narrow. It does not try to retrieve the best memory; it only tries to answer whether the *current* memory is still the right one. This makes it fast and cheap while protecting against the most common failure mode in conversational memory: staying focused on yesterday's topic when today's conversation has moved on.

> ***In plain terms:*** *The first question hippomem asks before doing anything else is: "Are we still talking about the same thing?" If the answer is a confident yes, it uses what it already has. Only if the answer is unclear or no does it look further.*

### 3.3 C2: Local Scan and Scoring

If C1 does not resolve the turn, hippomem scores all active and dormant working memories against the current query. This is the local scan — a targeted search within the memories that are already close to hand.

The query is constructed by concatenating the current message with the last few turns of conversation history. This is then embedded into a vector, and each candidate memory's stored vector is retrieved (from FAISS if available, re-embedded on the fly if not). The composite relevance score for each memory is computed as:

```
score = (0.5 × semantic_similarity) + (0.3 × relevance_score) + (0.2 ×
recency_bias)
```

Where:

- `semantic_similarity` is the cosine similarity between the query embedding and the memory's stored vector, mapped to [0, 1] via `(cos(θ) + 1) / 2`
- `relevance_score` is the memory's current decay-adjusted relevance score (starts at 1.0, decays with each passing hour)
- `recency_bias` is a linear decay over the past 168 hours (one week): 1.0 at 0 hours, 0.5 at 168 hours, 0.0 beyond approximately 336 hours

The top 3 active and top 2 dormant memories are selected by score. If the highest score across all candidates meets the local confidence threshold (default 0.6), C2 is declared high-confidence and the cascade stops. C3 is skipped.

> ***In plain terms:*** *C2 scores your active and recently-used memories against what you just said. It considers how similar the memory is to the current question, how relevant it has been historically, and how recently it was last used. If a good enough match is found here, the expensive full-database search never happens.*

### 3.4 C3: Long-Term Hybrid Retrieval

C3 is triggered when C2 does not find a confident local match — typically because the topic has shifted, a session is just beginning, or the relevant context is older than what working memory holds. This stage searches the full long-term memory store.

C3 combines three retrieval mechanisms:

**Dense vector search (FAISS).** The query is embedded and searched against the full FAISS index of all stored memory vectors using approximate nearest-neighbor search. This captures semantic similarity across the entire history.

**Keyword retrieval (BM25).** A BM25 index is built over the text content (`core_intent` + `updates`) of all episodic memories. Retrieval is done by tokenizing the query, stemming, and filtering stopwords. BM25 captures cases where exact terminology matters — technical terms, names, acronyms — that may not translate reliably into semantic vector space.

**Reciprocal Rank Fusion (RRF).** The ranked outputs of FAISS and BM25 are merged into a single list using Reciprocal Rank Fusion:

```
rrf_score(m) = Σ  1 / (k + rank(m, list))
```

For each retrieval list in which memory `m` appears, its rank (1-indexed) is used. The parameter `k = 60` (configurable) controls rank sensitivity. A memory appearing in both lists receives an additive boost from both terms. The merged list is sorted by RRF score descending.

After fusion, the top-5 primary results are retrieved from the database and scored using the same composite formula as C2.

**Graph expansion.** Each primary result is used as a seed for one hop of graph traversal. Neighboring memories — connected by any navigational edge with weight ≥ 0.1 — are retrieved as additional candidates. This allows memories that are *structurally adjacent* to a strong match to surface, even if they did not appear in vector or lexical search. A memory about a project kickoff may bring along its temporal successor — the first sprint review — through the graph, even if that successor did not match the query directly. Up to 5 graph-expanded memories are added to the final result set.

The merged primary and graph-expanded results are re-ranked by composite score and returned.

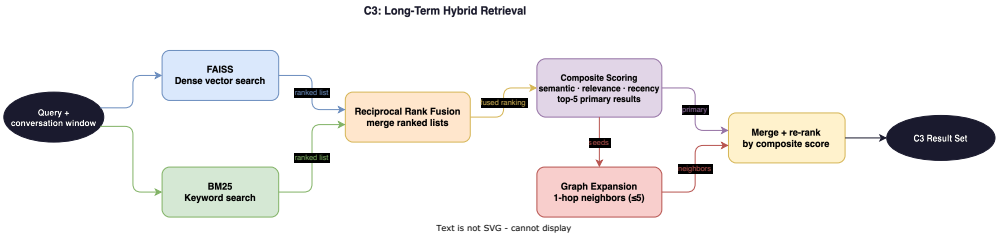



Figure 4 — C3 long-term hybrid retrieval

*Figure 4 — C3 long-term hybrid retrieval (simplified)*

> ***In plain terms:*** *When the quick local search doesn't find a good match, hippomem does a deeper search using two complementary strategies — one that finds similar ideas (FAISS) and one that finds matching words (BM25) — then combines them. It also follows connections between memories, so that finding one relevant memory can bring along related ones even if they didn't match the original question directly.*

### 3.5 Context Synthesis

After the cascade collects candidate memories — from whichever stage resolved the turn — they are not passed directly to the application LLM. Instead, a synthesis step produces a compact, coherent memory context paragraph.

A synthesis LLM call receives the candidate memories (episodic and entity), the current user message, and — if self-memory is enabled — the user's identity profile (persona or formatted traits). This makes the output identity-aware: the synthesis LLM is not working from a blank slate about who the user is, which shapes how it frames and prioritizes the context it produces. It produces:

- `synthesized_context`: a single coherent paragraph summarizing the relevant memory context, written to be injected directly into the system prompt
- `events_used`: which memories actually contributed, and each one's role (primary, supporting, or associative)
- `reasoning`: a brief explanation of why those memories were chosen

The `events_used` list is resolved back to the original engram UUIDs, which are stored in `DecodeResult.used_engram_ids`. These UUIDs flow directly into `encode()` and are the mechanism by which the encoder knows which memories were involved in this turn.

The final context is wrapped as `## Memory Context\n\n{text}` and returned to the application as `result.context`, ready to be injected into the LLM system prompt.

If synthesis fails for any reason, a deterministic fallback concatenates the `core_intent` strings of the candidate memories — ensuring `decode()` never fails silently and always returns something useful.

> ***In plain terms:*** *Rather than dumping raw memory records into the prompt, hippomem writes a clean, coherent paragraph explaining what is relevant about the current conversation. It also tracks exactly which memories were used — so the subsequent update step knows what to reinforce.*

---

## 4. retrieve(): Direct Programmatic Search

`decode()` is designed for one specific job: take the current conversational moment, find what is relevant, and produce a synthesized context string ready to inject into an LLM prompt. It does this well, but it is an opinionated pipeline — it makes LLM calls, it synthesizes, it returns a formatted string.

`retrieve()` is the lower-level counterpart. It gives applications direct access to the memory store without any of the synthesis overhead. No LLM calls are made. The result is not a paragraph — it is a structured, hierarchical collection of episodes, each carrying its linked entities and graph-connected neighbors. Applications that want to build their own processing on top of memory, power a search UI, or query memory independently of a conversation turn, use `retrieve()` for this.

One important difference from `decode()`: `retrieve()` does not degrade gracefully. If something goes wrong, the exception propagates to the caller. `decode()`, by contrast, will fall back to an empty context string rather than letting a retrieval failure break the LLM call it is serving.

### 4.1 Mode-Driven Primary Retrieval

The `mode` parameter controls which search strategy is used to find primary candidates from the full memory store. Three modes are available:

**`faiss`** embeds the query and searches the full FAISS index using approximate nearest-neighbor search. This captures semantic similarity — it finds memories that mean something similar to the query, even if they use different words. If the FAISS index is empty or unavailable, `faiss` mode returns an empty result immediately rather than erroring.

**`bm25`** takes a different approach: it tokenizes and stems the query, then runs BM25 over the text content (`core_intent + updates`) of all episodic memories. BM25 is a keyword-based algorithm — it is good at finding exact matches for technical terms, names, and acronyms that may not translate reliably into embedding space. When a user asks about a specific project name or a precise technical concept, BM25 is often the better searcher.

**`hybrid`** runs both FAISS and BM25 and then merges their ranked outputs using Reciprocal Rank Fusion (RRF). The intuition behind RRF is straightforward: a memory that appears near the top of both lists is a stronger match than one that only appears in one.

After merging, candidates are filtered to EPISODE and SUMMARY kinds with non-null `core_intent`. Each candidate is then scored using the same composite formula used in C2 and C3:

```
score = (w_sem × cosine) + (w_rel × relevance_score) + (w_rec × recency_bias)
```

Default weights: `w_sem=0.5, w_rel=0.3, w_rec=0.2` (overridable via the `w_*` parameters). For memories that only appeared in BM25 results, `cosine` is 0.0. The top `top_k` candidates are collected in merged-rank order, then re-sorted by composite score descending.

> ***In plain terms:*** *You can think of the three modes as different search strategies — FAISS for finding conceptually similar memories, BM25 for finding exact keyword matches, and hybrid for getting the best of both. Hybrid is usually the right default. The composite score then re-ranks whatever each strategy found, factoring in not just relevance to the query but also how important and how recent each memory is.*

### 4.2 Per-Episode Enrichment

Once the primary episodes are ranked, the result is not just a flat list of matching memories. Each primary episode is independently enriched with two additional layers of context: the entities linked to that episode, and related episodes reachable through the memory graph.

**Entities (`entities[]`).** The encoder, when it created or updated each episode, wrote MENTION edges from the episode to any named entities discussed in that turn. During retrieval, those MENTION edges are followed to load the entity profiles attached to each primary episode. Entities are sorted by their role in that episode — protagonists (the main subject) before incidental references — and then by reinforcement count descending within the same role tier. The result is capped at `entity_count` per episode.

**Related episodes (`related_episodes[]`).** The memory graph connects episodes to each other through navigational edges (SIMILARITY, TEMPORAL, RETRIEVAL, TRIADIC). One hop of graph traversal is performed from each primary episode, retrieving all neighbors with edge weight ≥ 0.1. MENTION links are excluded — traversal is restricted to episode-to-episode connections. These neighbors are sorted by edge weight descending, scored using the same composite formula as primary results, and capped at `graph_count`. Crucially, the semantic score for these neighbors reuses the FAISS vector already in memory — no new embedding call is made. If a neighbor's vector is absent, its cosine

contribution is 0.0. Related episodes do not carry their own entities or further related episodes — enrichment does not recurse.

Both enrichment steps are independently fault-tolerant. A failure in entity loading or graph traversal for any individual episode returns `[]` for that field and does not affect the rest of the result.

The most important distinction between `retrieve()` and `decode()` is the output format. `decode()` produces something ready to hand to an LLM — a clean prose paragraph that summarizes what the system knows. `retrieve()` produces something ready to hand to code — a structured object that an application can inspect, filter, display, or process however it needs.

> ***In plain terms:*** *`retrieve()` doesn't just hand you a list of matching memories. Each memory comes packaged with the people and projects it references, and with the memories that are most closely connected to it through the graph. You get a small neighborhood around each result, not just the result itself.*

---

## 5. Encoding: The encode() Path

After the application's LLM has generated its response, `encode()` is called. This closes the memory loop: hippomem decides how the completed turn should affect the memory store.

The encoder receives the user message, the assistant response, and the `DecodeResult` from the prior step. The `DecodeResult` carries the `used_engram_ids` — the UUIDs of memories that actually appeared in synthesis — and this is the primary input to the routing decision.

Figure 5 — The encode() path

*Figure 5 — The encode() path (simplified)*

### 5.1 Routing: Reinforcement vs. Creation

The encoder's first decision splits the work into two fundamentally different paths:

**Path A** — the `used_engram_ids` list from `decode()` is non-empty, meaning memories were actively used in synthesis this turn. The encoder works *with* those memories: reinforcing them, updating their content, or branching away from them when the conversation has meaningfully diverged.

**Path B** — no prior memories were used. The encoder decides whether to create a new memory, buffer a trace, or skip the turn entirely.

This routing is what prevents hippomem from blindly creating a duplicate memory on every turn. A system without Path A would fragment what should be a single evolving memory into dozens of slightly-different records of the same topic.

### 5.2 Path A: Drift Detection and Memory Update

Path A begins by asking whether the current turn still belongs to the active memory's topic. This is **drift detection** — a single LLM call that examines the active memory's `core_intent`, its existing fact updates, and the recent turns of conversation, returning one of two decisions:

- **`update_existing`**: the conversation is still on the same topic. Proceed to update the memory's content.
- **`create_new_branch`**: the conversation has meaningfully diverged. Create a new episodic engram for the new thread.

**When updating:** The encoder calls an LLM to extract what this turn added to the existing memory. It may append a new fact to the `pending_facts` buffer (setting `needs_consolidation=True`), refine the `core_intent` if the topic has evolved, or both. The `updates` list — the clean consolidated baseline — is not touched by the encoder; it is only written by the consolidation step. Content changes are detected via SHA-256 hashing of `core_intent + updates` — re-embedding only happens when content has actually changed, avoiding unnecessary API calls. The used engrams are also promoted to the head of the active working memory list, and their `reinforcement_count` and `last_updated_at` are bumped.

**When branching:** A new episodic engram is created for the divergent thread. A temporal graph edge is written from the prior active memory to the new one, preserving the narrative chain. The new memory becomes the head of the active list.

> ***In plain terms:*** *When you are continuing an existing topic, hippomem updates the memory rather than creating a duplicate. If the conversation takes a significant turn, it starts a new memory and links it to the old one — like opening a new chapter while keeping a reference to the one before it.*

### 5.3 Path B: Memory Creation and the Trace Buffer

When no prior memories were involved, the encoder must decide from scratch what to do with the current turn.

**The creation decision** is made by an LLM call (`should_create`) that evaluates the current turn in the context of existing ETS traces and recent conversation history. The question is: does this turn establish something that should be remembered? A meaningful new topic, a concrete project discussion, a significant preference expressed for the first time — these warrant a full episodic engram. A quick question-and-answer, a clarification, a pleasantry — these do not.

If creation is warranted, a new episodic engram is created: the LLM generates a `core_intent` and initial `updates` list, the engram is embedded and added to the FAISS index, and it is prepended to the active working memory list.

If creation is not warranted, a second LLM call (`maybe_append_to_ets`) asks whether the turn at least deserves a trace in the ETS. Traces are lightweight — short text summaries stored without embedding — and serve as context for future memory creation decisions within the session. If neither creation nor tracing is warranted, the turn is skipped entirely.

> ***In plain terms:*** *Not everything said in a conversation needs to become a permanent memory. The encoder makes a deliberate decision each turn. Significant new topics become full memories. Minor exchanges become traces — lightweight notes that might inform future memory creation. Everything else is let go.*

### 5.4 Entity Extraction and Linking

After the episodic path completes — regardless of which branch was taken — the encoder runs entity extraction. This step is independent of the episodic outcome: entities are extracted and updated even on turns that produced no new episodic memory.

**Entity reinforcement** runs first, before any extraction LLM call. For each entity UUID in `used_entity_ids` from the decode step, the encoder bumps `reinforcement_count` and `last_updated_at`. This strengthens the signal for entities the decoder already confirmed were relevant to the current turn, without re-running any LLM logic.

**Entity extraction** then calls an LLM with the user message, assistant response, and recent conversation history. The LLM returns a list of significant named entities, each with a canonical name, entity type, mention role, and extracted facts.

For each significant entity, the encoder applies the following identity lookup strategy:

1. **Name scan.** All existing entity engrams of the same type are queried and matched by name. Matches are classified as `exact` (case-insensitive equality), `substring` (one name contains the other), or `token` (names share at least one meaningful word).
2. **0 candidates → create new.** A new entity engram is created with the extracted facts placed in `pending_facts` (and `updates` starting empty, to be filled by the first consolidation run).
3. **Single exact match → auto-update.** If there is exactly one exact name match, facts are appended to `pending_facts` immediately. No LLM call is needed.
4. **Synthesis hint shortcut.** If the name scan finds candidates, the encoder checks which of those candidates appeared in `used_entity_ids` from the decode step. If exactly one candidate is a known entity from synthesis, facts are appended to it directly — no disambiguation LLM call is needed. The decode step's entity set acts as a strong prior for identity resolution.
5. **LLM disambiguation.** If there are multiple candidates or only non-exact matches (and no clear synthesis hint), an LLM call is made. The candidates are shown with their names and known facts, and the LLM selects the best match or returns null.
6. **Create new.** If no match is found or the LLM returns null, a new entity engram is created.

Once the entity is identified, a **MENTION edge** is written from the episode engram to the entity engram, with a `mention_type` (protagonist, subject, or referenced) indicating the entity's role in that episode. This edge makes it possible to surface entities during retrieval: any episode engram that the cascade selects can bring along its associated entities.

On turns where no episode was created (ETS or skip), entities are still extracted and updated, but no MENTION link is written — there is no episode to link to.

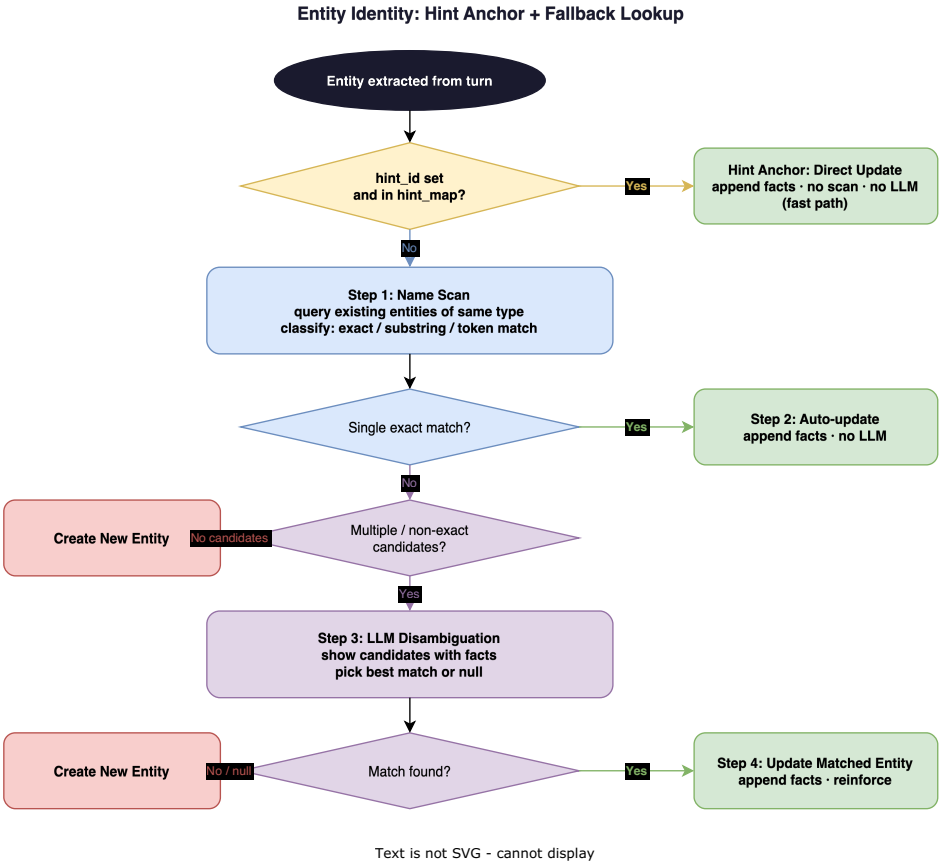



Figure 8 — Entity identity: lookup strategy

*Figure 8 — Entity identity: name scan → auto-update → synthesis hint → disambiguation → create*

> ***In plain terms:*** *Entity identity resolution works in tiers of confidence. An exact name match updates the right entity immediately. If the entity was already surfaced during memory retrieval this turn, that prior confirmation acts as a shortcut — the encoder already knows who it is. Only ambiguous cases, where multiple entity records could plausibly match, go to an LLM to decide. When nothing matches, a new entity profile is created.*

### 5.5 Self-Trait Accumulation

The final step of every encode turn is self-trait extraction. This runs regardless of which episodic path was taken — a turn that produced no episodic memory can still yield self-trait signals.

An LLM call receives the user's current active trait profile and extracts any signals from the current turn that are absent from or meaningfully changed in that profile. The LLM assigns a `confidence_estimate` to each signal using a three-tier rubric:

- **High (0.8–1.0)**: explicitly stated facts, action-significant preferences, or explicit directives ("always", "never", "remember", "make sure you"). These are written as immediately active.
- **Medium (0.6–0.8)**: clearly implied but not directly stated. Stored but not yet active — they activate on a second independent observation.
- **Below 0.6**: skipped entirely. Not written to the database.

A special rule applies to explicit instructions: any phrasing that contains "remember", "always", "never", or "make sure you" is assigned `confidence_estimate = 0.9+` regardless of how the preference is worded, and mapped to the `preference` category.

The accumulation logic is a straightforward upsert:

- **New key**: a `SelfTrait` row is inserted with `evidence_count=1` and `confidence_score` set to the LLM's estimate. If `confidence_estimate >= 0.8`, `is_active=True` immediately. Otherwise `is_active=False` — the trait exists but is not yet used.
- **Existing key, value changed**: the LLM returns the complete final merged value (not a delta). The prior value is preserved in `previous_value`, the new value replaces it, and `evidence_count` and confidence are updated. One level of value history is maintained.
- **Existing key, inactive**: if `evidence_count` reaches 2, or if the new observation has `confidence_estimate >= 0.8`, `is_active` is set to `True`.
- **Reactivation**: if a trait was previously deactivated by consolidation, any new observation immediately sets `is_active=True` again.

Confidence accumulates toward a cap of 1.0. Deactivation is the exclusive responsibility of the consolidation layer — a trait is never deactivated by the encoder.

> ***In plain terms:*** *Every conversation is also an opportunity to learn a little more about the user. hippomem quietly notes preferences, goals, and context from things said naturally, without requiring anyone to fill out a profile. Strong signals go active immediately; weaker signals are held in reserve until a second mention confirms them. Only rarely — when something hasn't been mentioned in a long time and was never strongly reinforced — does the system quietly let it fade.*

---

## 6. Memory Over Time: Consolidation

`consolidate()` is periodic maintenance. It can be triggered manually or run on a schedule via the background task. Four steps run in order, each independently fault-tolerant:

1. **Episode fact consolidation** — runs when episode consolidation is enabled (default: on)
2. **Entity profile enrichment** — runs when entity extraction is enabled
3. **Self-trait pruning** — runs when self memory is enabled
4. **Persona generation** — runs when self memory is enabled

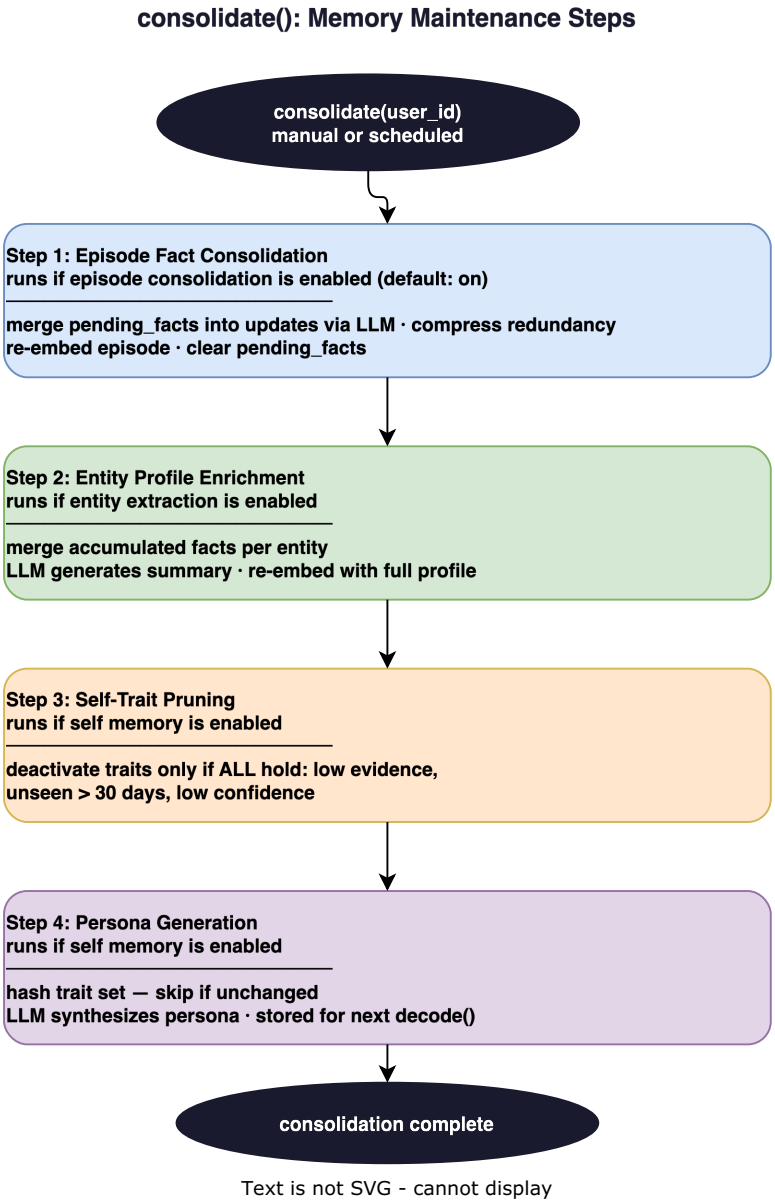



Figure 10 — consolidate() maintenance steps

*Figure 10 — consolidate() maintenance steps (simplified)*

## 6.1 Episode Fact Consolidation

As conversations continue, episodic engrams accumulate update statements in their `pending_facts` buffer. Each new significant fact extracted from a turn is appended there, and the `needs_consolidation` flag is set to `True`. The clean consolidated baseline (`updates`) is left untouched between consolidation runs.

When consolidation runs, all episode engrams flagged with `needs_consolidation=True` are processed. For each, an LLM call merges the existing consolidated updates with the pending ones — superseding contradictions, compressing redundancy, and capping the merged list at approximately 8–10 items. The result replaces `updates`, `pending_facts` is cleared, and the episode is re-embedded so the FAISS index reflects the latest content.

This compression matters for long-lived topics. A project that a user discusses across dozens of sessions would otherwise accumulate a growing unbounded list of update statements. Consolidation keeps the episode representation concise and retrieval-quality high over time.

> ***In plain terms:*** *Episodes grow as conversations return to the same topic. Periodically, hippomem compresses that growing list of updates into a clean, deduplicated summary — so retrieval stays sharp even for memories that have been visited many times.*

### 6.2 Entity Profile Enrichment

Entity engrams accumulate new facts in `pending_facts` as the encoder processes turns where those entities are mentioned. The `updates` field — the clean consolidated baseline — is not touched until consolidation runs.

When consolidation runs, all entity engrams flagged with `needs_consolidation=True` are processed. The LLM receives two explicitly labeled sections:

- **Consolidated facts** — the trusted baseline from prior consolidation runs, treated as authoritative
- **New facts** — the pending buffer accumulated since last consolidation, to be integrated

This separation ensures the LLM treats the consolidated baseline as reliable and focuses its integration effort on what is genuinely new. The output is a merged fact list and a fresh `summary_text` narrative summarizing everything known about the entity. The entity is then re-embedded so future semantic search reflects the full enriched profile. Once processed, `pending_facts` is cleared and `needs_consolidation` is set to `False`.

Entity decay runs at a much slower rate than episodic memories (`decay_rate ≈ 0.999/hour`). What is known about a person should not fade as quickly as the memory of a specific conversation.

> ***In plain terms:*** *Periodically, hippomem merges scattered entity facts into a clean summary and updates the search representation to reflect everything learned. It knows the difference between what it already confirmed in a prior pass and what arrived since — and treats them accordingly.*

### 6.3 Self-Trait Pruning

Before persona synthesis, consolidation runs a conservative pruning pass over the user's active self-traits. A trait is deactivated only when **all three** conditions hold simultaneously:

```
evidence_count < 2              (seen only once — never reinforced)
last_observed_at > 30 days ago  (not mentioned in a month)
confidence_score < 0.7          (never achieved high confidence)
```

The AND logic is intentionally strict. A high-confidence single-shot preference ("always use bullet points") survives because `confidence >= 0.7`. A frequently-reinforced trait (`evidence_count >= 2`) also survives regardless of recency. Only genuinely weak, stale, and rarely-observed traits are deactivated. A deactivated trait is immediately reactivated the next time it is observed in an encode turn.

### 6.4 Persona Generation

When self memory is enabled, consolidation synthesizes all active self-traits into a single **Persona engram**.

Process: 1. Load all `is_active=True` traits. Activation is the single quality gate — no additional confidence filter is applied here. 2. Compute a deterministic hash of the current trait set (content + values). If hash matches the existing persona's stored hash, skip — traits haven't changed. Avoids redundant LLM calls. 3. LLM generates a structured identity summary from the active traits, organized

into named categories with bold Markdown headers. The recommended order (skipping categories with no traits): **Professional Context**, **Goals**, **Projects**, **Communication Preferences**, **Constraints**, **Personality**, **Relationships**. The `Communication Preferences` section includes all explicit always/never directives verbatim. Length is proportional to the traits present — there is no word limit. 4. Persona is stored or updated as a Persona engram with an explicit `updated_at` timestamp.

**Pending trait injection.** The Persona engram is injected into the synthesis prompt on each `decode()` call. However, new traits may be observed between consolidation runs. To ensure the decoder always has fresh self-knowledge, `_load_self_profile()` checks whether any active trait has `last_observed_at > persona.updated_at`. If so, those traits are appended to the persona as a `**Recently observed (pending consolidation):**` block, and the decode source is marked `persona+pending` rather than `persona`. Once consolidation runs and incorporates those traits, the pending block disappears.

> ***In plain terms:*** *After accumulating enough self-knowledge, hippomem writes a structured profile of the user — organized by category, not limited to a sentence or two — and uses it to make retrieval identity-aware. Traits observed between consolidation runs are still visible to the decoder immediately, so fresh signals are never lost while waiting for the next consolidation pass.*

---

## 7. The Memory Graph

hippomem's memory store is not a flat collection of vectors. Memories are connected to each other through a typed, weighted graph that is built passively during normal encode operations and used actively during long-term retrieval.

This graph is one of the properties that distinguishes hippomem from simpler memory approaches. Nearest-neighbor search finds the most similar thing to what you asked. Graph traversal finds things that are *related to* what you found — and sometimes those are the memories that actually matter.

### 7.1 Edge Types and Semantics

Five edge types are defined, each representing a distinct kind of relationship between memories:

| Edge Type | Created by | Weight delta | Semantic meaning |
|---|---|---|---|
| **SIMILARITY** | Any encode with a new or updated embedding | +0.1 | Two memories are semantically close in vector space |
| **TEMPORAL** | Memory branching (drift or cold-create) | +0.15 | One memory is a narrative successor to another |
| **RETRIEVAL** | Two or more memories used together in synthesis | +0.15 | Two memories were co-relevant in context |
| **TRIADIC** | FAISS neighbors of a memory share a SIMILARITY link | +0.02 | Structural clustering bonus — strengthens triangles |
| **MENTION** | Entity extraction | (no weight) | An episode references an entity (directional) |

The four navigational edge types (SIMILARITY, TEMPORAL, RETRIEVAL, TRIADIC) are stored in canonically sorted order for bidirectional deduplication. Edge weights accumulate — the same pair of memories receiving the same edge type repeatedly will develop an increasingly strong connection.

MENTION edges are directional (episode → entity) and carry no weight. They exist to enable entity injection during synthesis: the decoder loads the entities linked to whichever episode engrams were shortlisted, bringing entity profiles into context alongside the episodic memories.

### 7.2 Graph-Augmented Retrieval

During C3 retrieval, the top-5 primary results serve as seeds for graph expansion. For each seed, the graph is traversed one hop to retrieve all neighboring memories connected by any navigational edge with weight ≥ 0.1 (MENTION links are excluded from traversal). Up to 5 graph-expanded memories are added to the result set, scored using the same composite formula as primary results.

The practical effect: finding a memory surfaces its neighborhood. A memory about a project kickoff can bring along the sprint review through TEMPORAL. A memory that was frequently retrieved together with another can bring it through RETRIEVAL. Semantically similar memories surface through SIMILARITY. The graph makes the memory store behave like a network rather than a bag of vectors.

Graph expansion is configurable: it can be disabled, the minimum edge weight can be adjusted, and the maximum number of graph-expanded candidates can be set.

> ***In plain terms:*** *Finding a relevant memory is like finding a city on a map. Once you find the city, you can also see the roads out of it. Graph expansion follows those roads — surfacing connected memories that might also be relevant, even if they didn't match the original query directly.*

---

## 8. Operational Architecture

### 8.1 In-Process vs. Daemon Mode

hippomem can be used in two modes.

**In-process mode** embeds hippomem directly in the application as a Python library. `MemoryService` is instantiated, setup is called once, and `decode()` / `encode()` / `consolidate()` are called directly. This is the simplest integration path — no additional processes, no network overhead, full access to all configuration options.

```
async with MemoryService(llm_api_key="sk-...", llm_base_url="...", config=MemoryC
        onfig()) as memory:
    result = await memory.decode(user_id, message, conversation_history=history)
    # inject result.context into your LLM system prompt
    response = await your_llm_call(message, memory_context=result.context)
    await memory.encode(user_id, message, response, decode_result=result)
```

**Daemon mode** runs hippomem as a standalone server process (`hippomem serve`). Applications connect to it as a client via HTTP using `HippoMemClient`. This is useful when: - Multiple applications or services share a single memory instance - Memory should persist independently of the application's lifecycle - The Studio UI and Inspector are needed without embedding them in the application

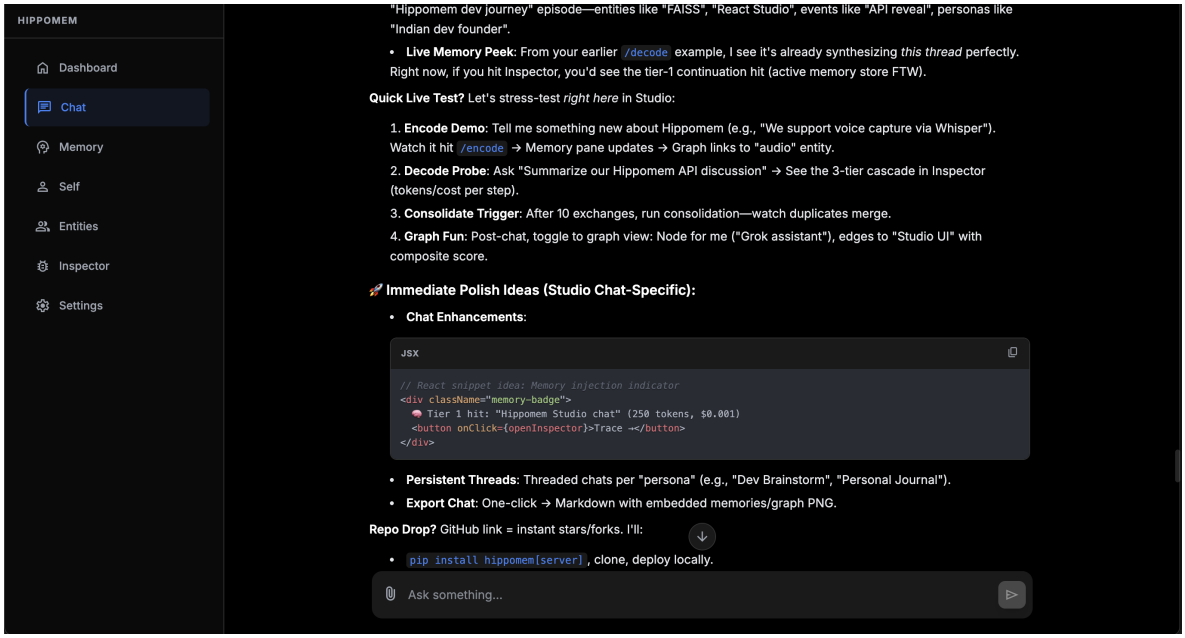
```
# Application code (separate process from the hippomem daemon)
client = HippoMemClient(base_url="http://localhost:8719")
result = await client.decode(user_id, message, conversation_history=history)
await client.encode(user_id, message, response, decode_result=result)
# Direct retrieval (also available via the client):
search = await client.retrieve(user_id, query, mode="hybrid", top_k=5)
```

Both modes share the same underlying memory engine. The daemon adds an HTTP API layer (FastAPI) and the Studio UI on top. The database, FAISS index, and all memory representations are identical between modes.

## 8.2 Studio: The Human Interface

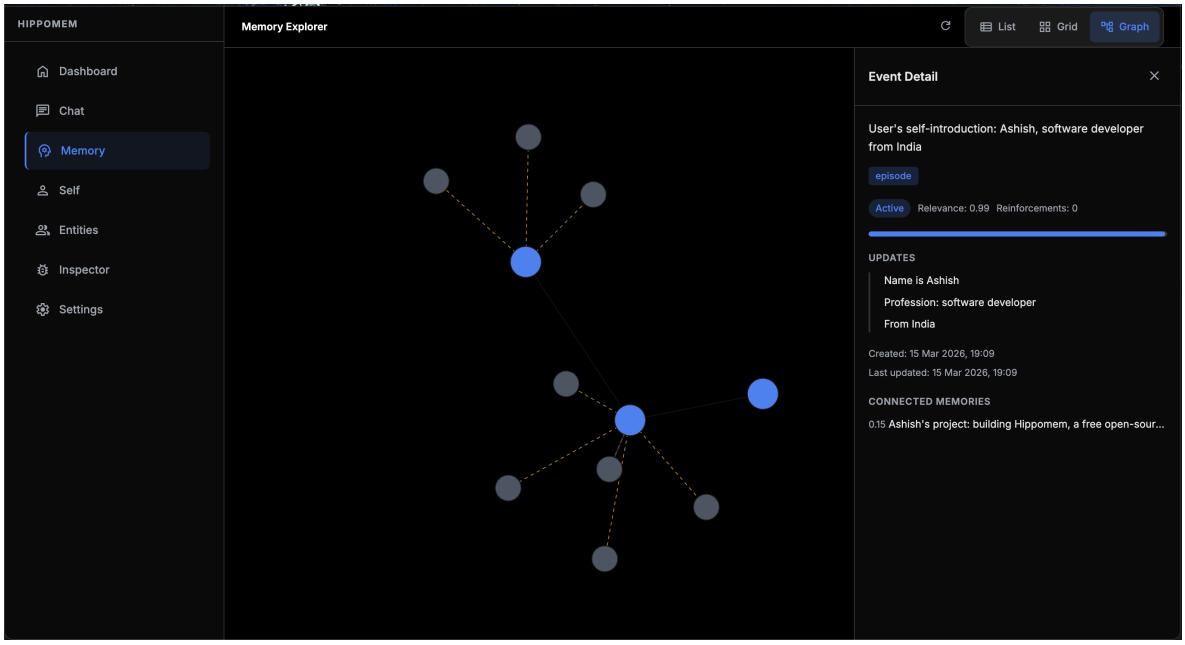
hippomem ships with a built-in web application called **Studio**, served by the daemon at `http://localhost:8719`. Studio is designed for two audiences: developers who need to inspect and understand memory behavior, and end users who want to interact with memory directly — without writing code.

**Chat.** Studio provides a built-in chat interface with full streaming support. Each turn shows which decode stage was used (C1, C2, or C3), how many memories were involved, and the live progress of decode and encode steps as they happen. Chat history is persisted to the database and loads automatically when Studio is opened — conversations survive server restarts.



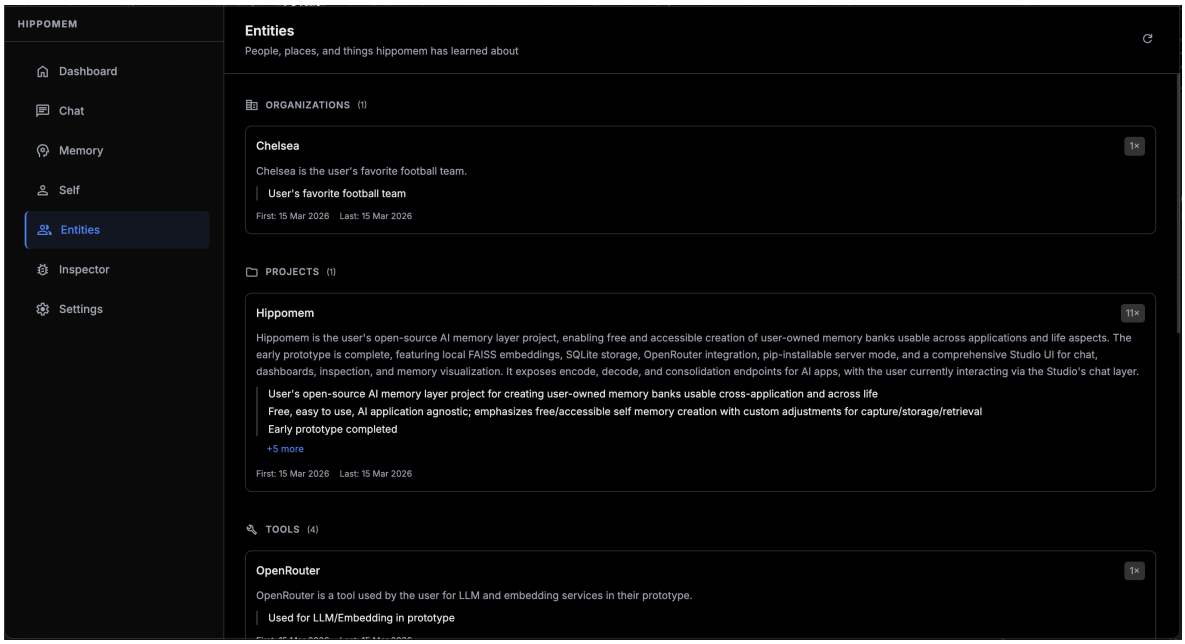

*Studio chat interface*

**Memory Graph.** A visual, interactive graph of all stored memories and their connections. Nodes are colored by kind (episodes, entities, persona). Edges are shown with their type and weight. Clicking a node reveals the full memory detail — its topic, fact updates, relevance score, reinforcement count, and connected neighbors.

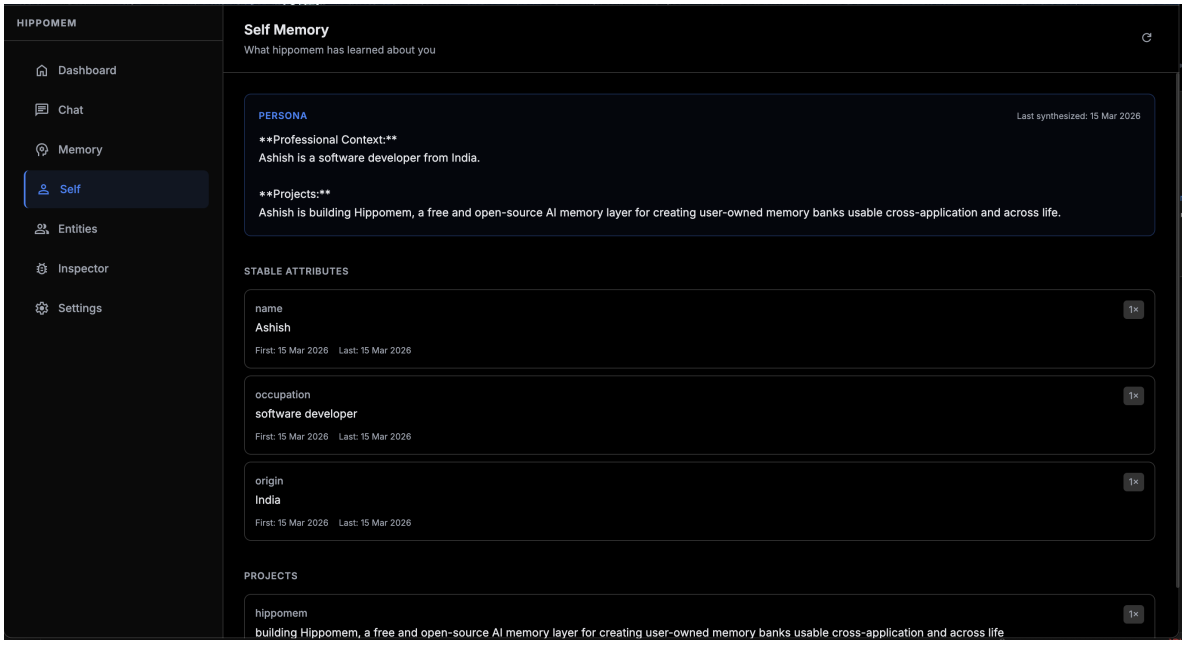



*Studio memory graph — episodes, entities, and connections*

**Memory Explorer.** Browsable views of episodic memories, entity profiles, self-traits, and the persona narrative. Each view shows the current state of every memory, including its metadata.

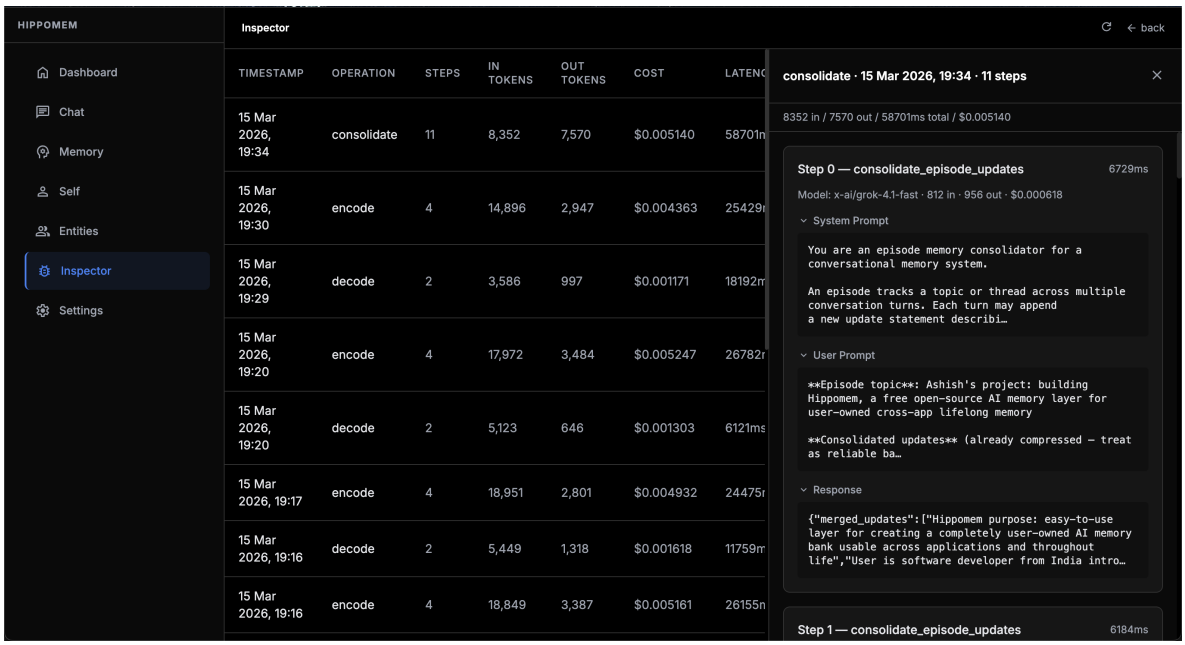



*Studio memory explorer — entity profiles*



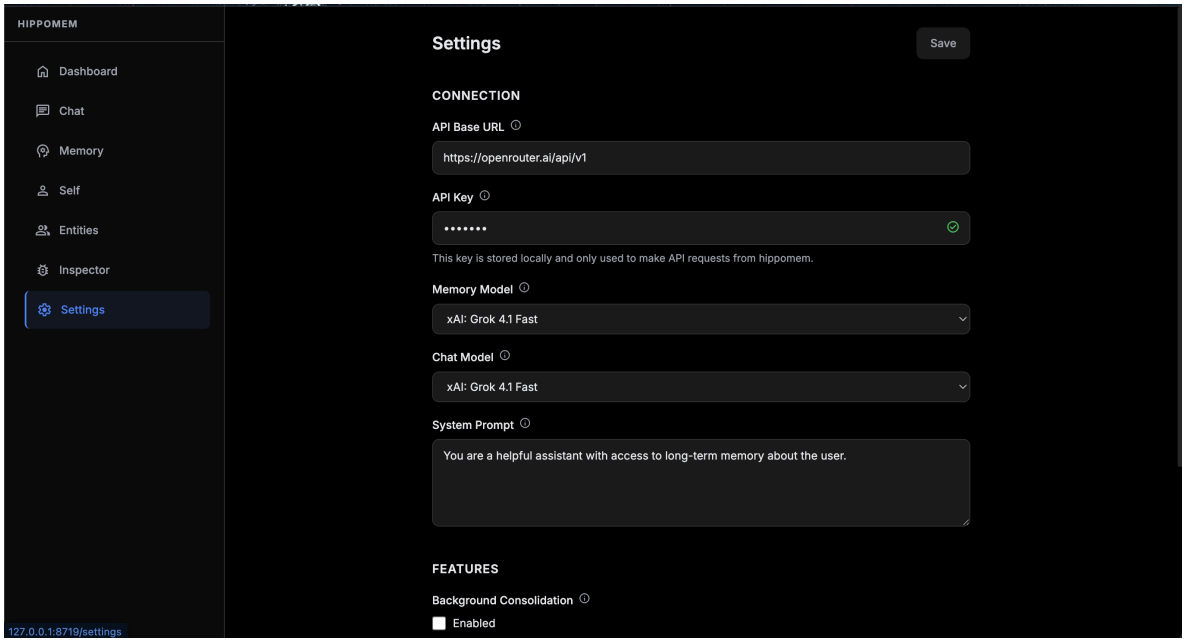

*Studio memory explorer — self-traits and persona*

**Inspector.** Every LLM call made during decode, encode, and consolidate is logged with its full prompt, response, token counts, cost, and latency. The Inspector allows developers to see exactly what happened in any given turn — which memories were retrieved, what the synthesis LLM was given, what drift detection concluded, and what entity disambiguation decided.




*Studio Inspector — full LLM call log with tokens, cost, and latency*

**Settings.** All configuration parameters — model selection, cascade thresholds, feature flags, and decay rates — are exposed in a settings UI. Changes take effect immediately: most parameters are hot-reloadable without a restart, and credentials are warm-reloadable.




*Studio Settings — configuration, model selection, and feature flags*

> ***In plain terms:*** *You do not need to read code to use hippomem. Studio is a browser-based application where you can have conversations, watch memory being built in real time, explore everything the system has learned, and inspect every decision it made — all without touching a terminal.*

### 8.3 Observability and Inspection

Every top-level operation (`decode()`, `encode()`, `consolidate()`) is recorded as an `LLMInteraction` row in the database, aggregating token counts, cost, latency, and the operation's output (which memories were used, what action was taken, etc.).

Each individual LLM call within that operation is recorded as an `LLMCallLog` row — a full capture of the prompt messages, raw response, tokens, cost, latency, and an ordering index within the operation. This enables both high-level cost analysis ("how expensive is decode on average?") and low-level debugging ("exactly what was the continuation check prompt and what did the model say?").

**Turn linking.** `decode()` and `encode()` for the same conversational turn share a `turn_id`. This makes it possible to query all operations that happened within a single user turn — the decode retrieval, the synthesis, the content update, the entity extraction — as a single unit.

**Conversation turn storage.** Each completed turn (user message + assistant response + the memory context injected into that turn) is persisted to the database as a `ConversationTurn` record. Each turn is linked to the engrams that were involved: decoded engrams (surfaced and injected into context) and the encoded engram (the episode the turn was written into). This bidirectional linkage enables reverse lookup: given an engram UUID, retrieve every raw conversation turn where that memory played a role. This is the foundation for future memory explainability features — showing not just what a memory contains, but what conversations produced it.

**Progress streaming.** When using the daemon, each phase of decode and encode emits a step label via Server-Sent Events as it begins. The Studio chat interface uses this to show live progress. Applications using `HippoMemClient` can subscribe to the same stream.

**Configuration at runtime.** The server supports two classes of configuration change: - **Hot reload**: threshold values, feature flags, decay rates, and capacity parameters are applied in-place on the running

`MemoryService` and take effect on the next operation. - **Warm reload**: model names and credentials require re-initializing the underlying `LLMService` and `EmbeddingService` objects, but do not require a server restart.

---

## 9. Limitations and Trade-offs

**LLM-dependent decisions.** Continuation detection, drift detection, update extraction, entity extraction, entity disambiguation, and self-trait extraction are all LLM calls. These judgments are probabilistic and can be wrong. Every LLM call has a fallback, but accuracy is bounded by model quality and prompt quality. Cheaper or less capable models will have lower accuracy at these decision points.

**Graph noise.** Graph expansion improves recall but connections accumulate from usage patterns that may not reflect genuine semantic relevance. Weak or overly dense graphs can surface adjacent rather than actually relevant memories. The configurable minimum edge weight threshold and maximum graph events per C3 call exist to manage this, but the problem is not fully eliminated.

**Local-first architecture.** All data is stored in SQLite with FAISS indices as local files. Appropriate for individual users and small teams; not the same as distributed infrastructure for large-scale multi-tenant workloads. Scaling to that model is a future direction.

**Evaluation is ongoing.** Quantitative evaluation — latency by cascade path, memory quality over long interaction sequences, retrieval precision and recall at different turn counts — is planned as the next phase of work. This paper describes the architecture as designed; empirical performance claims will follow separately.

---

## 10. Conclusion

The challenge of memory in AI systems is not primarily a retrieval challenge. Retrieval is tractable. The harder challenge is treating memory as something that evolves — something that should be reinforced when it remains relevant, updated when the topic continues, branched when it diverges, and gradually deprioritized when it becomes stale. A system that only stores and retrieves is not the same as a system that actually remembers.

hippomem proposes a practical answer to this challenge. Its central contribution is not any single component but the coherence of the whole: a staged retrieval cascade optimized for the common case, an encoding layer that makes deliberate decisions about how each turn affects stored memory, a consolidation process that maintains memory quality over time, and a graph that makes the memory store navigable beyond nearest-neighbor search. These pieces work together to produce a memory system that behaves less like a database and more like an accumulating understanding.

The system is early. There is real work ahead — evaluation, scaling, the deeper entity and knowledge layers described in the roadmap. But the foundation is functional and the design is sound. For assistants, copilots, and agent-like systems that need more than short-context prompting, hippomem offers memory infrastructure: a subsystem that helps LLM applications decide what matters now, what should persist later, and how those two concerns should stay aligned as interactions accumulate.

The project is open. Contributions, questions, and criticism are welcome.

---

# Appendices

## Appendix A: Retrieval Cascade Decision Table

| C1 decision | C1 confidence | C2 high-confidence | C3 triggered | cascade label | Events sent to synthesis |
|---|---|---|---|---|---|
| CONTINUE | ≥ threshold (0.7) | (skipped) | No | C1 | active[0] only |
| CONTINUE | < threshold | True | No | C2 | top 3 active + top 2 dormant |
| SHIFT or UNCERTAIN | — | True | No | C2 | top 3 active + top 2 dormant |
| SHIFT or UNCERTAIN | — | False | Yes, with results | C3 | C2 events + graph/FAISS/BM25 results |
| (no active memories) | — | False | Yes, with results | C3 | FAISS + BM25 + graph results only |
| (no active memories) | — | False | No results | C2 | (empty → no LLM synthesis call) |

---

## Appendix B: Scoring Formulas Reference

**Composite retrieval score (C2 and C3)**

```
score = (0.5 × semantic_similarity) + (0.3 × relevance_score) + (0.2 ×
recency_bias)

semantic_similarity = (cosine(query, memory) + 1) / 2        mapped to [0, 1]
recency_bias        = max(0, min(1, 1.0 - (hours_since / 168) × 0.5))
                                                              linear decay over
~1 week
```

Default weights: semantic 0.5, relevance 0.3, recency 0.2. All three weights are configurable.

**Relevance decay (applied on every encode turn)**

```
score(t) = score(t₀) × 0.98 ^ hours_elapsed
```

~2% per hour, ~40% per day. Rate is configurable via `decay_rate_per_hour`. Decay runs during encoding, not as a periodic consolidation step.

**Working memory demotion**

Capacity-driven and FIFO. When the active tier is full (`max_active_events`, default 5) and a new memory is created, the least-recently-used active memory is pushed to dormant. When the dormant tier is full (`max_dormant_events`, default 5), the oldest dormant memory is evicted from fast access (remains in long-term storage). No score formula involved.

**Reciprocal Rank Fusion (C3 FAISS + BM25 merge)**

```
rrf_score(m) = Σ  1 / (k + rank(m, list))     k = 60 (configurable)
```

Summed over all lists in which memory `m` appears. Rank is 1-indexed.

---

## Appendix C: Memory Graph Edge Semantics

| Edge type | Directional | Canonical sort | Weight delta | Created by |
|---|---|---|---|---|
| SIMILARITY | No | Yes (sorted UUIDs) | +0.1 per co-embedding | `process_links_realtime()` |
| TEMPORAL | No | Yes | +0.15 per branch | `strengthen_temporal_links()` |
| RETRIEVAL | No | Yes | +0.15 per co-synthesis | `strengthen_retrieval_links()` |
| TRIADIC | No | Yes | +0.02 per triangle | `process_links_realtime()` |
| MENTION | Yes (episode → entity) | No | 0 (no weight) | `_link_entity_to_episode()` |

MENTION links carry a `mention_type`: `protagonist` (entity is the main subject), `subject` (key participant), or `referenced` (incidental mention). During entity injection in synthesis, this type is used to prioritize which entities to include when space is limited.

---

## Appendix D: MemoryConfig Quick Reference

Key configuration fields and their defaults:

| Parameter | Default | What it controls |
|---|---|---|
| `max_active_events` | 5 | Working memory active tier capacity |
| `max_dormant_events` | 5 | Working memory dormant tier capacity |
| `ephemeral_trace_capacity` | 8 | ETS size per session |
| `decay_rate_per_hour` | 0.98 | Relevance decay rate |
| `continuation_threshold` | 0.7 | C1 confidence required to skip C2/C3 |
| `local_scan_threshold` | 0.6 | C2 confidence required to skip C3 |
| `retrieval_semantic_weight` | 0.5 | Composite score: semantic component |
| `retrieval_relevance_weight` | 0.3 | Composite score: relevance component |
| `retrieval_recency_weight` | 0.2 | Composite score: recency component |
| `enable_graph_expansion` | True | C3 graph expansion on/off |
| `enable_bm25` | True | BM25 keyword search on/off |
| `enable_entity_extraction` | True | Entity memory on/off |
| `enable_self_memory` | True | Self-trait and persona on/off |
| `enable_episode_consolidation` | True | Episode update compression on/off |
| `enable_background_consolidation` | False | Automatic background consolidation |
| `consolidation_interval_hours` | 1.0 | Background consolidation interval |

---

## Appendix E: Integration Quick Reference

**In-process (Python library)**

```python
from hippomem import MemoryService, MemoryConfig

async with MemoryService(
    llm_api_key="sk-...",
    llm_base_url="https://openrouter.ai/api/v1",
    config=MemoryConfig(),
) as memory:
    # Before your LLM call:
    result = await memory.decode(
        user_id="user_123",
        message=user_message,
        conversation_history=history,  # list of (user, assistant) pairs
    )
    # Inject result.context into your system prompt, then call your LLM.
    # After your LLM responds:
    await memory.encode(
        user_id="user_123",
        user_message=user_message,
        assistant_response=assistant_response,
        decode_result=result,           # pass the full DecodeResult
        conversation_history=history,
    )
    # Periodically (or on a schedule):
    await memory.consolidate("user_123")

    # Direct retrieval (independent of decode/encode):
    result = await memory.retrieve(
        user_id="user_123",
        query="What did we discuss about the product launch?",
        mode="hybrid",      # "faiss" | "bm25" | "hybrid"
        top_k=5,
        entity_count=4,     # linked entities per episode
        graph_count=5,      # graph-neighbor episodes per primary result
    )
    for episode in result.episodes:
        print(episode.core_intent, [e.core_intent for e in episode.entities])
```

### Daemon mode (client/server)

```bash
# Start the daemon (once):
hippomem serve --port 8719
```

```python
from hippomem.client import HippoMemClient

client = HippoMemClient(base_url="http://localhost:8719")
result = await client.decode(user_id, message, conversation_history=history)
await client.encode(user_id, message, response, decode_result=result)
# Direct retrieval via the client:
search = await client.retrieve(user_id, query, mode="hybrid", top_k=5)
```

---